Synopsys® Cognitive Services

AI Gateway Docker Installation Guide

March 2026

SYNOPSYS®

Copyright and Proprietary Information Notice

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at https://www.synopsys.com/company/legal/trademarks-brands.html.
https://www.synopsys.com/company/legal/trademarks-brands.html. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content. www.synopsys.com

Contents

Synopsys AI Gateway Installation Document

The following procedure outlines the steps for installing the AI Gateway using Docker Compose.

Pre-Installation Checklist

Before beginning the installation, ensure you have the following information and resources ready:

Required Information

- [ ] LLM Provider details (OpenAI, Azure OpenAI, AWS Bedrock, etc.)
- [ ] API Keys for each provider
- [ ] Provider endpoints and regions
- [ ] Model names and deployment IDs
- [ ] Port numbers for AI Gateway service
- [ ] SSL/TLS certificates (if enabling HTTPS)

System Requirements Verification

- [ ] Docker Engine version 27.4.0 or later
- [ ] Docker Compose version 2.32.0 or later
- [ ] Minimum 16 GB RAM (or higher preferred)
- [ ] Minimum 4 CPU cores (or higher preferred)
- [ ] At least 1 GB available disk space
- [ ] Docker privileges to run commands

Network Requirements

- [ ] Outbound internet access to LLM provider endpoints
- [ ] Available port for AI Gateway (default: 8084)
- [ ] Firewall rules allowing traffic to/from AI Gateway

Access Requirements

- ☐ Access to Synopsys Installer
- ☐ Read/write permissions in installation directory

Getting and Using the Synopsys Installer

The Synopsys Installer is required to install the AI Gateway package.

Step 1: Download Prerequisites

1. **Download the Synopsys Installer:**
   - Contact your Synopsys representative or access the Synopsys download portal
   - Download `SynopsysInstaller_v5.9.run` (or latest version)
   - Ensure you have the latest version of the installer
2. **Download AI Gateway SPF files:**
   - Download the following files for your version:
     - `scs_ai_gateway_docker_compose_v<version>_linux64.spf`
     - `scs_ai_gateway_docker_compose_v<version>_common.spf`
   - Place all SPF files in a single directory (e.g., `~/downloads/0.0.83/`)

Step 2: Install the Synopsys Installer

```
# Make the installer executable
chmod +x SynopsysInstaller_v5.9.run

# Run the installer
./SynopsysInstaller_v5.9.run

# When prompted, specify installation directory:
# Please specify installation directory [.]: <your_path>/installer
```

Example output:

```
Installing Synopsys Installer 5.9 into the directory
'/home/user/installer'...
Creating installation directory '/home/user/installer'...
  Unpacking: SynopsysInstaller.tgz ...
Installation complete.
```

Step 3: Install AI Gateway using Synopsys Installer

```
# Navigate to your working directory
cd ~

# Run the installer
<installer_path>/installer/installer
```

You will see the following prompts:

1. **Source directory prompt:**

   ```
   Enter the path to the source directory containing the downloaded
   EFT file(s): /path/to/spf/files
   ```

   Enter the directory containing your SPF files.

2. **Installation directory prompt:**

   ```
   Enter the full path to the directory where you want to
   install Synopsys products: /home/user/scs_ai_gateway
   ```

   Enter your desired installation path.

3. **Installation summary:**

```
############### Installation Summary ################
Product:    scs_ai_gateway_docker_compose
Release:    0.0.83
Platforms:  linux64 common
Disk Space: 34 MB
Target Dir:
/home/user/scs_ai_gateway/scs_ai_gateway_docker_compose/0.0.83
#####################################################
Accept, Install? [yes]: yes
```

Step 4: Navigate to Installed Files

After installation, the AI Gateway files will be in a nested directory structure:

```
# The installer creates this structure:
<install_dir>/
└── scs_ai_gateway_docker_compose/
    └── 0.0.83/
        ├── admin/
        ├── install.log
        └── scs_ai_gateway_docker_compose/
            ├── docker-compose.yaml
            ├── scs-ai-gateway-0.0.83.tar
            └── config/
                └── gateway-config.yaml

# Navigate to the actual AI Gateway directory:
cd
<install_dir>/scs_ai_gateway_docker_compose/0.0.83/scs_ai_gateway_do
cker_compose/
```

Note: The directory structure has multiple levels. The actual Docker Compose files are in: `<install_dir>/scs_ai_gateway_docker_compose/<version>/scs_ai_gateway_docker_compose/`

Docker Prerequisites

- Docker Engine version 27.4.0 or later
- Docker Compose version 2.32.0 or later
- Docker privileges on the host machine to run Docker commands

Docker Hardware Requirements

- Minimum 16 GB of RAM (or higher is preferred)
- Minimum 4 CPU cores (or higher is preferred)
- Minimum 1 GB of disk space (or higher is preferred)

Upgrading from a Previous Version (Existing Installation)

If you already have AI Gateway installed and just want to move to a newer version, you can skip the installer steps and reuse your existing `config/gateway-config.yaml` and any certificates. Perform only the following minimal actions:

> Upgrade in 3 Quick Steps
>
> 1. Load the new image tarball (replace `<latest-version>`):
>
> ```
> docker load -i scs-ai-gateway-<latest-version>.tar
> ```
>
> 2. Update (or create) your `.env` with the new tag (adjust path only if it changed):
>
> ```
> GATEWAY_IMAGE_PATH=llm.local/ai-gateway
> GATEWAY_IMAGE_TAG=<latest-version>
> ```
>
> (Leave other values—admin keys, provider keys, ports—unchanged unless release notes say otherwise.) 3. Redeploy the container so it picks up the new image:
>
> ```
> docker compose down
> docker compose up -d
> ```
>
> (Or simply `docker compose up -d` if the old container isn't running.)

Validation (recommended):

```
docker compose ps
curl -s http://localhost:8084/liveness
```

If the release notes mention configuration schema changes, review and update `config/gateway-config.yaml` before restarting.

Docker Installation Steps

1. **Locate the installed AI Gateway files:**

After running the Synopsys Installer (see previous section), navigate to the installation directory:

```
# Navigate to the AI Gateway directory
# Note: The path includes version number and nested directories
cd
<install_dir>/scs_ai_gateway_docker_compose/<version>/scs_ai_gate
way_docker_compose/

# Example:
cd
~/scs_ai_gateway/scs_ai_gateway_docker_compose/0.0.83/scs_ai_gate
way_docker_compose/
```

The AI Gateway package contains the following directory structure:

```
scs_ai_gateway_docker_compose/
|   ├── docker-compose.yaml            # Docker Compose
configuration
|   ├── scs-ai-gateway-<version>.tar   # Docker image
|   ├── .env.example                   # Example environment
variables
|   ├── config/                        # Configuration directory
|   |     ├── gateway-config.yaml     # AI Gateway configuration
|   ├── certs/                         # Certificate directory
|   |     ├── gencerts.sh              # Certificate instructions
|   ├── provider-config-examples/      # Provider configuration
examples
|   |     ├── openai-example.yaml
|   |     ├── azure-openai-example.yaml
|   |     ├── aws-bedrock-example.yaml
|   |     └── ...
|   ├── validation-scripts/            # Validation helper
scripts
|   |     ├── test_gw_health.sh
|   |     ├── test_gw_routes.sh
|   |     ├── test_upstream.sh
|   |     └── test_chat_completion.sh
|   └── QUICKSTART.md                  # Quick installation
reference
```

2. Import the Docker image from the `scs-ai-gateway-<version>.tar` file. For example:

   ```
   docker load -i scs-ai-gateway-<version>.tar
   ```

   This decompresses the contents of the tar file and loads the image into your local Docker environment.

3. **Create and configure the environment file:**

   Copy the example environment file and update with your values:

```
cp .env.example .env
```

Edit the `.env` file with your configuration:

```
# Docker Image Configuration
GATEWAY_IMAGE_PATH=llm.local/llm-gateway
GATEWAY_IMAGE_TAG=<version>

# Port Configuration
EXPOSED_HOST_PORT=8084
HTTP_PORT=8080

# Gateway Admin Keys (minimum 32 characters)
ADMIN_ACCESS_KEY=your-admin-key-min-32-chars
CHAT_API_ACCESS_KEY=your-chat-api-key-min-32-chars

# Provider API Keys
AZURE_API_KEY=your-azure-api-key
OPENAI_API_KEY=your-openai-api-key
AWS_ACCESS_KEY=your-aws-access-key
AWS_SECRET_KEY=your-aws-secret-key
AWS_REGION=us-east-1
GCP_PROJECT=your-gcp-project
GCP_REGION=us-central1
GCP_API_KEY=your-gcp-api-key

# Optional: TLS Configuration
HTTP_CERT_PEM_FILE=
HTTP_KEY_PEM_FILE=

# Optional: Downstream CA Certificates
DOWNSTREAM_CA_CERTS=
```

4. **Generate SSL/TLS Certificates (Optional):**

   If you need SSL/TLS certificates for secure connections, the AI Gateway package includes a convenient script to generate self-signed certificates:

```
# Navigate to the certs directory
cd certs/

# Run the certificate generation script
sh gencerts.sh
```

This will generate the following files:

```
certs/
├── ca.key          # CA private key
├── ca.pem          # CA certificate
├── server.crt      # Server certificate
├── server.csr      # Certificate signing request
├── server.key      # Server private key
└── gencerts.sh     # Certificate generation script
```

Note: For production environments, use certificates from a trusted Certificate Authority (CA) instead of self-signed certificates.

5. **Configure the AI Gateway:**

   Update the `gateway-config.yaml` file to configure routes and providers:

   ⚠ **Important Configuration Guidelines:**

   - **DO modify:** API keys, route names, provider configurations, endpoints
   - **DO NOT modify:** The docker-compose.yaml file unless you're an advanced user
   - Use provider examples as templates:

```
# List available provider examples
ls provider-config-examples/

# Copy a provider example as starting point
cp provider-config-examples/azure-openai-example.yaml
config/gateway-config.yaml

# Edit the configuration
vi config/gateway-config.yaml
```

6. **Review docker-compose.yaml (DO NOT MODIFY unless necessary):**

   The docker-compose.yaml is pre-configured. Here's what it contains for reference:

```
name: llm-gateway
services:
  backend:
    image: ${GATEWAY_IMAGE_PATH}:${GATEWAY_IMAGE_TAG}
    volumes:
      - ${PWD}/config/gateway-config.yaml:/app/config.yaml

    environment:
      # (REQUIRED) Gateway configuration type - currently only
file-based configuration is supported
      - LLM_GATEWAY_FILE_BASED=${FILE_BASED_CONFIG:-true}
      # (OPTIONAL) Gateway log level
      - LOG_LEVEL=${LOG_LEVEL:-info}
      # (OPTIONAL) Public key PEM file path for TLS encryption,
to enable HTTPS
      - HTTP_CERT_PEM_FILE=${HTTP_CERT_PEM_FILE:-}
      # (OPTIONAL) Private key PEM file path for TLS encryption,
to enable HTTPS
      - HTTP_KEY_PEM_FILE=${HTTP_KEY_PEM_FILE:-}
      # (OPTIONAL) HTTP port on which the AI Gateway runs inside
the container
      - HTTP_PORT=${HTTP_PORT:-8080}
      # (REQUIRED) Gateway configuration file path
      - LLM_GATEWAY_FILE_PATH=/app/config.yaml
      # (REQUIRED) Super admin access key
      - LLM_GATEWAY_ADMIN_ACCESS_KEY=${ADMIN_ACCESS_KEY:-root}
      # (REQUIRED) Super admin access key for the chat API
      - LLM_GATEWAY_CHAT_API_ACCESS_KEY=${CHAT_API_ACCESS_KEY:-
root}
      # (OPTIONAL) Google service account JSON file
    ports:
      - "${EXPOSED_HOST_PORT:-8084}:${HTTP_PORT:-8080}"
    restart: unless-stopped
```

7. **Deploy the AI Gateway using Docker Compose:**

   Start the AI Gateway service:

```
docker compose -f docker-compose.yaml up -d
```

This creates and starts the containers in detached mode.

Common Docker Compose Commands:

```
# Start services in foreground (see logs)
docker compose up

# Start services in background
docker compose up -d

# View logs
docker compose logs -f

# Stop services
docker compose down

# Restart services (after config changes)
docker compose restart

# View running containers
docker compose ps

# Pull latest images
docker compose pull

# Validate docker-compose.yaml
docker compose config
```

8. **Validate the Installation:**

   Use the provided validation scripts:

```
# Make scripts executable
chmod +x validation-scripts/*.sh

# Test gateway health
./validation-scripts/test_gw_health.sh localhost:8084

# Test available routes
./validation-scripts/test_gw_routes.sh localhost:8084
$ADMIN_ACCESS_KEY

# Validate upstream endpoints
./validation-scripts/test_upstream.sh
```

Or manually check the service health:

```
curl -i --location 'https://localhost:8084/liveness'
```

Expected response:

```
HTTP/1.1 200 OK
{"status":"healthy","timestamp":"..."}
```

9. **Test AI Gateway functionality:**

   Run the chat completion test script or use curl:

```
curl -i --location 'https://localhost:8084/api/{route-
name}/chat/completions' \
-H 'Content-Type: application/json' \
-H 'x-api-key: ${GATEWAY_API_KEY}' \
--data '{
    "messages": [
        {
            "role": "user",
            "content": "What is Deep Learning?"
        }
    ]
}'
```

Or use the test script:

```
./validation-scripts/test_chat_completion.sh localhost:8084
<route-name> $GATEWAY_API_KEY
```

AI Gateway Service Configuration Using Docker Compose

Gateway Configuration File Structure

The AI Gateway is configured using a YAML file (`gateway-config.yaml`) that defines API keys, routes, and providers. The `gateway-config.yaml` file contains the following main sections:

> IMPORTANT (v2025.11.01+): OpenAI and Huggingface provider configurations MUST use the base URL + suffix format (no full endpoint URLs). Provide only the base, e.g. `https://api.openai.com/v1`, and rely on default suffixes or set custom ones via `chat-completions-suffix`, `embedding-suffix`, `rerank-suffix`. Update configs before installing/upgrading to v2025.11.01+.

Gateway API Keys

- The `gateway-api-keys` section contains the API keys for the gateway. Each key can be given permissions for specific routes.

- Permissions can include:
  - `text_completion`
  - `chat_completion`
  - `embedding`
  - `rerank`

Example:

```
gateway-api-keys:
  - name: openai-api-key
    key: <set a minimum 32-bit alphanumeric value>
    resources:
      routes:
        - name: azure-openai-gpt4o
          permissions:
            - chat_completion
        - name: openai-gpt4-5
          permissions:
            - chat_completion
```

Routes

- Routes are abstractions for a given LLM model and provider.
- Each route is associated with one or more providers.

Example:

```
routes:
  - name: azure-openai-gpt4o
    providers:
      - az-openai-gpt4o
  - name: openai-gpt4-5
    providers:
      - oai-gpt4-5
```

Providers

- Providers are services that provide access to specific LLM models.
- Supported provider types include:
  - `openai` (Standard OpenAI integration)
  - `passthrough` (Direct passthrough mode)
  - `proxy` (Generic proxy mode)
  - `aws-bedrock` (AWS Bedrock service)
  - `azure-openai` (Azure's OpenAI service)
  - `azure-ai` (Azure AI service)
  - `nvidia-nims` (Nvidia NIMS service)
  - `huggingface` (Hugging Face models)
  - `gcp-genai` (Google Cloud Platform Generative AI)
  - `gcp-vertex` (Google Cloud Platform Vertex AI)

Provider configuration varies by type. See the sample below for details.

Sample gateway-config.yaml

```
gateway-api-keys:
  - name: openai-api-key
    key: <set a minimum 32-bit alphanumeric value>
    resources:
      routes:
        - name: azure-openai-gpt4o
          permissions:
            - chat_completion
        - name: openai-gpt4-5
          permissions:
            - chat_completion

routes:
  - name: azure-openai-gpt4o
    providers:
      - az-openai-gpt4o
  - name: openai-gpt4-5
    providers:
      - oai-gpt4-5

providers:

  az-openai-gpt4o:
    type: azure-openai
    provider-config:
      azure-openai-config:
        endpoint: azureopenai.openai.azure.com
        deployment-id: GPT-4o
        api-version: 2024-10-21
        model:
          name: gpt-4o
          modes:
            - chat_completion
            - text_completion
        auth:
          api-key:
            key: ${AZURE_API_KEY}

  oai-gpt4-5:
    type: openai
    provider-config:
```

```
      headers:
        include:
          Authorization: Bearer ${BEARER_TOKEN}
      openai-config:
        # Base URL only; default /chat/completions suffix auto-
applied
        url: https://api.openai.com/v1
        # To override add: chat-completions-suffix:
/custom/chat/path
        model:
          name: gpt-4.5
          modes:
            - chat_completion
        auth:
          api-key:
            key: ${OPENAI_API_KEY}
  # Hugging Face
  huggingface:
    type: huggingface
    provider-config:
      request_params:
        timeout: 60
      headers:
        include:
          Authorization: Bearer <your_huggingface_token>
      huggingface-config:
        # Base model URL; defaults applied: /v1/chat/completions,
/v1/embeddings, /rerank
        url: https://api-inference.huggingface.co/models/nomic-
embed-text-v1-5-bce8a
        # Optional overrides:
        # chat-completions-suffix: /models/nomic-embed-text-v1-5-
bce8a/v1/chat/completions
        # embedding-suffix: /v1/embeddings
        # rerank-suffix: /v1/rerank
        model:
          name: nomic-embed-text-v1-5-bce8a
          modes:
            - chat_completion
            - embedding
            - rerank
```

```
  # AWS
  bedrock-sonnet-3-5-v1:
    type: aws-bedrock
    provider-config:
      bedrock-config:
        region: ${AWS_REGION}
        model:
          name: anthropic.claude-3-5-sonnet-20240620-v1:0
          modes:
            - chat_completion
        auth:
          access-key:
            access-key: ${AWS_ACCESS_KEY}
            secret-key: ${AWS_SECRET_KEY}
  # GCP Vertex
  gcp-vertex-gemini-2-5-flash:
    type: gcp-vertex
    provider-config:
      gcp-vertex-config:
        project-id: ${GCP_PROJECT}
        project-location: ${GCP_REGION}
        model:
          name: gemini-2.5-flash-preview-04-17
          modes:
            - chat_completion
        auth:
          service-account:
            # Path to service_account.json
            filepath: ${GOOGLE_SERVICE_ACCOUNT_KEY}
  # GCP Gemini
  gcp-genai-gemini15:
    type: gcp-genai
    provider-config:
      gcp-genai-config:
        endpoint: https://generativelanguage.googleapis.com
        model:
          name: gemini-1.5-pro-002
          modes:
            - chat_completion
            - text_completion
        auth:
```

```
        api-key:
          key: ${GCP_API_KEY}
```

V1 API Suffix Configuration (Required for v2025.11.01+)

For OpenAI and Huggingface providers (only) the gateway now derives operation endpoints by appending suffixes to the base URL.

Default suffixes:

- OpenAI: `/chat/completions`, `/embeddings`
- Huggingface: `/v1/chat/completions`, `/v1/embeddings`, `/rerank`

Customization keys (inside `openai-config` / `huggingface-config`):

- `chat-completions-suffix`
- `embedding-suffix`
- `rerank-suffix` (Huggingface only)

Migration example:

```
# Old (invalid in v2025.11.01+):
url: https://api.openai.com/v1/chat/completions

# New (base + default suffix):
url: https://api.openai.com/v1

# New with custom suffix:
url: https://api.openai.com/v1
chat-completions-suffix: /llm/chat
```

Increase Client Timeouts in AI Gateway

By default, the AI Gateway client waits 30 seconds for a response from the backend provider. If the provider takes longer than 30 seconds, the AI Gateway will terminate the

connection and return a 500 Internal Server Error. To increase the timeout for a specific provider, use the following configuration:

```
providers:
  gcp-genai-gemini15:
    type: gcp-genai
    provider-config:
      request_params:
        timeout: 60 # Timeout in seconds
      gcp-genai-config:
        endpoint: https://generativelanguage.googleapis.com
        model:
          name: gemini-1.5-pro-002
          modes:
            - chat_completion
            - text_completion
        auth:
          service-account:
            # Path to service_account.json
            filepath: ${GOOGLE_SERVICE_ACCOUNT_KEY}
```

Admin Keys Configuration

AI Gateway provides APIs to view all the routes added to the configuration and also a way to override authentication for all providers.

```
name: llm-gateway
services:
  backend:
    image: ${GATEWAY_IMAGE_PATH}:${GATEWAY_IMAGE_TAG}
    volumes:
      - ${PWD}/config/gateway-config.yaml:/app/config.yaml

    environment:
      # (REQUIRED) Super admin access key
      - LLM_GATEWAY_ADMIN_ACCESS_KEY=${ADMIN_ACCESS_KEY:-root}
      # (REQUIRED) Super admin access key for the chat API
      - LLM_GATEWAY_CHAT_API_ACCESS_KEY=${CHAT_API_ACCESS_KEY:-root}
```

- **LLM_GATEWAY_ADMIN_ACCESS_KEY**: Super admin access key to view all routes. This accepts a 32-character alphanumeric key.
- **LLM_GATEWAY_CHAT_API_ACCESS_KEY**: Super admin access key for the chat API. This key can be used to authenticate to all routes.

Add Self-Signed or Custom Client Certificates

AI Gateway supports external CA certificates to establish secure connections. The service will load the files mentioned in the `DOWNSTREAM_CA_CERTS` environment variable at runtime.

Mount the certificates to the container and add `DOWNSTREAM_CA_CERTS` with the file locations. For multiple files, use the `;` delimiter.

```
name: llm-gateway
services:
  backend:
    image: ${GATEWAY_IMAGE_PATH}:${GATEWAY_IMAGE_TAG}
    volumes:
      - ${PWD}/certs/:/app/certs/
    environment:
      -
DOWNSTREAM_CA_CERTS=/app/certs/file1.pem;/app/certs/file2.crt
```

Configure Google Service Account-Based Authentication

Unlike the Google Gemini service, Google Vertex APIs do not support API key-based authentication. For Google Vertex, you need to mount the service account JSON file and add an environment variable to specify the path. This way, the AI Gateway service will pick up the credential JSON file during API authentication.

Mount the service account JSON file to the container:

```
name: llm-gateway
services:
  backend:
    image: ${GATEWAY_IMAGE_PATH}:${GATEWAY_IMAGE_TAG}
    volumes:
      - ${PWD}/creds/:/app/creds/
    environment:
      - GOOGLE_SERVICE_ACCOUNT_KEY=/app/creds/key.json
      - GCP_SERVICE_ACCOUNT_KEY=/app/creds/key2.json
```

You can mount any number of key JSON files, but the environment variable names should be unique for each file.

Troubleshooting

Common Installation Issues

1. GATEWAY_IMAGE_PATH Variable Not Set

Error:

```
$ docker compose -f docker-compose.yaml up -d
WARN[0000] The "GATEWAY_IMAGE_PATH" variable is not set. Defaulting
to a blank string.
WARN[0000] The "GATEWAY_IMAGE_TAG" variable is not set. Defaulting
to a blank string.
unable to get image ':': Error response from daemon: invalid
reference format
```

Solution:

```
# Ensure .env file exists and is properly configured
ls -la .env

# Source the environment file
source .env

# Verify variables are set
echo $GATEWAY_IMAGE_PATH
echo $GATEWAY_IMAGE_TAG

# Or set them directly
export GATEWAY_IMAGE_PATH=llm.local/llm-gateway
export GATEWAY_IMAGE_TAG=<version>

# Then run docker compose
docker compose up -d
```

2. Port Already in Use

Error: `bind: address already in use`

Solution:

```
# Check what's using the port
sudo lsof -i :8084

# Either stop the conflicting service or change the port in .env
EXPOSED_HOST_PORT=8085  # Use a different port
```

3. Container Exits Immediately

Error: Container starts then immediately stops

Solution:

```
# Check container logs
docker compose logs backend

# Common causes:
# - Invalid gateway-config.yaml syntax
# - Missing required environment variables
# - File permission issues

# Validate YAML syntax
python -m yaml config/gateway-config.yaml

# Check file permissions
ls -la config/gateway-config.yaml
chmod 644 config/gateway-config.yaml
```

4. Cannot Connect to Provider

Error: `Failed to connect to upstream provider`

Solution:

- Verify Docker has internet access
- Check if proxy settings are needed:

```
# Add to docker-compose.yaml environment section
- HTTP_PROXY=${HTTP_PROXY}
- HTTPS_PROXY=${HTTPS_PROXY}
- NO_PROXY=${NO_PROXY}
```

- Validate provider endpoints and API keys
- Run: `./validation-scripts/test_upstream.sh`

5. Certificate Errors

Error: `x509: certificate signed by unknown authority`

Solution:

```
# Place CA certificates in certs directory
cp /path/to/ca-cert.pem certs/

# Update .env file
DOWNSTREAM_CA_CERTS=/app/certs/ca-cert.pem

# For multiple certificates
DOWNSTREAM_CA_CERTS=/app/certs/cert1.pem;/app/certs/cert2.pem

# Update docker-compose.yaml to mount certs
volumes:
  - ${PWD}/certs:/app/certs
```

6. Permission Denied Errors

Error: `permission denied` when accessing files

Solution:

```
# Fix file ownership
sudo chown -R $USER:$USER .

# Fix file permissions
chmod 644 config/gateway-config.yaml
chmod 644 .env
chmod 755 validation-scripts/*.sh
```

7. Memory or Resource Issues

Error: `Cannot allocate memory` or performance issues

Solution:

```
# Check Docker resource limits
docker system info | grep -E "Total Memory|CPUs"

# Increase Docker Desktop resources (if applicable)
# Docker Desktop > Settings > Resources

# Add resource limits to docker-compose.yaml
services:
  backend:
    deploy:
      resources:
        limits:
          memory: 4G
          cpus: '2'
```

Validation Checklist

After installation, verify:

- ☐ Container is running: `docker compose ps`
- ☐ No errors in logs: `docker compose logs`
- ☐ Health endpoint responds: `curl http://localhost:8084/liveness`
- ☐ Routes are configured: `curl http://localhost:8084/v1/routes -H "x-api-key: $ADMIN_ACCESS_KEY"`
- ☐ Can access providers: `./validation-scripts/test_upstream.sh`
- ☐ Chat completion works: `./validation-scripts/test_chat_completion.sh`

Getting Help

If issues persist:

1. Collect logs: `docker compose logs > gateway.log 2>&1`
2. Check configuration: `docker compose config > compose-config.yaml`
3. Get container details: `docker inspect scs_ai_gateway_docker_compose_backend_1`
4. Contact Synopsys support with the collected information

Glossary

API Key

- **API Key**: An opaque token used to authenticate a user or an application accessing the AI Gateway APIs.
- API clients need to pass the API key as `x-api-key` in the request header.

Route

- **Route**: An abstraction for a given LLM model and provider. It defines the API endpoint for accessing the LLM.

Provider

- **Provider**: A service that provides access to specific LLM models.
- The provider can be a cloud service like AWS Bedrock or Azure OpenAI, or on-premise LLM servers.